

1. CRIAR BASE DE DADOS

```
CREATE DATABASE IF NOT EXISTS loja_musica;
```

```
USE loja_musica;
```

2. CRIAR TABELAS

```
CREATE TABLE compositores (
```

```
    id_compositor INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nome VARCHAR(100) NOT NULL,
```

```
    nacionalidade VARCHAR(50)
```

```
);
```

```
CREATE TABLE formatos (
```

```
    id_formato INT AUTO_INCREMENT PRIMARY KEY,
```

```
    tipo_formato VARCHAR(20) NOT NULL
```

```
);
```

```
CREATE TABLE partituras (
```

```
    id_partitura INT AUTO_INCREMENT PRIMARY KEY,
```

```
    titulo VARCHAR(150) NOT NULL,
```

```
    preco DECIMAL(10, 2) NOT NULL,
```

```
    stock INT DEFAULT 0,
```

```
    id_compositor INT,
```

```
    id_formato INT,
```

```
    FOREIGN KEY (id_compositor) REFERENCES compositores(id_compositor),
```

```
    FOREIGN KEY (id_formato) REFERENCES formatos(id_formato)
```

```
);
```

3. INSERIR DADOS

`INSERT INTO` compositores (nome, nacionalidade) `VALUES` ('Bach', 'Alemao');

`INSERT INTO` formatos (tipo_formato) `VALUES` ('PDF Digital');

`INSERT INTO` partituras (titulo, preco, id_compositor, id_formato)

`VALUES` ('Concerto Brandemburgo', 25.00, 1, 1);

4. CONSULTAR DADOS (SELECT)

-- Todas as colunas de uma tabela

`SELECT * FROM` compositores;

`SELECT * FROM` formatos;

`SELECT * FROM` partituras;

-- Colunas especificas com condicao `WHERE`

`SELECT` id_partitura, titulo, preco `FROM` partituras

`WHERE` preco < 20;

-- `LIKE` para buscar titulos que começam com 'S'

`SELECT` id_partitura, titulo, preco `FROM` partituras

`WHERE` titulo `LIKE` 'S%' `OR` titulo `LIKE` 's%';

-- `ORDER BY` para ordenar do maior preco para o menor

`SELECT` titulo, preco `FROM` partituras

`ORDER BY` preco `DESC`;

5. ATUALIZAR DADOS (UPDATE)

-- Mudar preco de uma partitura especifica

UPDATE partituras

SET preco = 35.00

WHERE id_partitura = 5;

-- Adicionar ano de composicao a varias partituras

UPDATE partituras SET ano_composicao = 1722 WHERE id_partitura = 1;

UPDATE partituras SET ano_composicao = 1791 WHERE id_partitura = 4;

UPDATE partituras SET ano_composicao = 1824 WHERE id_partitura = 7;

6. APAGAR DADOS (DELETE)

-- Apagar partituras compostas entre 1750 e 1880

DELETE FROM partituras WHERE ano_composicao BETWEEN 1750 AND 1880;

7. ALTERAR ESTRUTURA DA TABELA

-- Adicionar coluna ano_composicao a tabela partituras

ALTER TABLE partituras ADD COLUMN ano_composicao INT;

8. JUNCOES DE TABELAS (INNER JOIN)

-- Juntar as tres tabelas: compositores, partituras e formatos

SELECT c.nome, p.titulo, f.tipo_formato

FROM compositores c

INNER JOIN partituras p ON p.id_compositor = c.id_compositor

INNER JOIN formatos f ON p.id_formato = f.id_formato;

-- Mesma consulta mas excluindo titulos com 'Sinfonia'

```

SELECT c.nome, p.titulo, f.tipo_formato
FROM compositores c
INNER JOIN partituras p ON p.id_compositor = c.id_compositor
INNER JOIN formatos f ON p.id_formato = f.id_formato
WHERE p.titulo <> 'Sinfonia';

```

9. FUNCOES DE AGREGACAO

-- Soma de todos os precos das partituras

```

SELECT SUM(preco) FROM partituras;

```

-- Preco maximo de cada compositor

```

SELECT c.nome, MAX(p.preco) AS preco_maximo
FROM compositores c
INNER JOIN partituras p ON p.id_compositor = c.id_compositor
GROUP BY c.nome;

```

-- Preco minimo de cada compositor com ano de composicao

```

SELECT c.nome, p.ano_composicao, MIN(p.preco) AS preco_minimo
FROM compositores c
INNER JOIN partituras p ON p.id_compositor = c.id_compositor
GROUP BY c.nome;

```

-- Quantidade de partituras por compositor

```

SELECT c.nome, COUNT(p.titulo) AS quantidade_partituras
FROM compositores c
INNER JOIN partituras p ON p.id_compositor = c.id_compositor
GROUP BY c.nome;

```

10. ARREDONDAMENTO

-- Media de stock por compositor arredondada para cima

```
SELECT c.nome, CEIL(AVG(p.stock)) AS media_stock_arredondada
FROM compositores c
INNER JOIN partituras p ON p.id_compositor = c.id_compositor
GROUP BY c.nome;
```

11. FILTRAR GRUPOS (HAVING)

-- Compositores com menos de 2 partituras

```
SELECT c.nome, COUNT(*) AS total_partituras
FROM compositores c
INNER JOIN partituras p ON p.id_compositor = c.id_compositor
GROUP BY c.nome
HAVING COUNT(*) < 2;
```

12. VIEWS (TABELAS VIRTUAIS)

-- Primeira view: informacao basica de partituras

```
CREATE VIEW vw_partituras AS
SELECT c.nome AS compositor, p.ano_composicao, p.titulo, p.preco
FROM compositores c
INNER JOIN partituras p ON p.id_compositor = c.id_compositor;
```

```
SELECT * FROM vw_partituras;
```

-- Segunda view: incluir formato das partituras

```
CREATE VIEW vw_formatos AS
SELECT c.nome AS compositor, p.titulo, p.preco, f.tipo_formato
FROM compositores c
```

`INNER JOIN` partituras p `ON` p.id_compositor = c.id_compositor

`INNER JOIN` formatos f `ON` p.id_formato = f.id_formato;

`SELECT * FROM` vw_formatos;

13. INDICES (OTIMIZACAO)

-- Ver como o MySQL procura os dados (antes do indice) `EXPLAIN`

`EXPLAIN SELECT * FROM` partituras `WHERE` preco > 30;

-- Criar indice na coluna preco para buscas mais rapidas

`CREATE INDEX` idx_preco `ON` partituras(preco);

-- Ver como o MySQL procura os dados (depois do indice) `EXPLAIN`

`EXPLAIN SELECT * FROM` partituras `WHERE` preco > 30;

14. VER ESTRUTURA DAS TABELAS

`DESCRIBE` compositores;

`DESCRIBE` formatos;

`DESCRIBE` partituras;

15. ORDEM CORRETA DAS CLAUSULAS

1 `SELECT` c.nome, `COUNT`(*) `AS` total

2 `FROM` compositores c

3 `INNER JOIN` partituras p `ON` p.id_compositor = c.id_compositor

4 `WHERE` p.preco > 10

5 `GROUP BY` c.nome

6 `HAVING COUNT`(*) > 1

7 `ORDER BY` total `DESC`;

16. BACKUP E RESTORE (NO TERMINAL)

-- Fazer backup da base loja_musica

```
mysqldump -u root -p loja_musica > loja_musica.sql
```

-- Restaurar backup

```
mysql -u root -p loja_musica < loja_musica_backup.sql
```

17. ENTRAR NA BASE DE DADOS DESDE À TERMINAL

```
cd C:\wamp64\bin\mysql\mysql(TAB mostra versão)\bin\
```

```
~ .\mysql -u root -p
```

PASSWORD: (ENTER)

18. DICAS IMPORTANTES COM SEUS DADOS

Sempre use **WHERE** em **UPDATE** e **DELETE**

Exemplo: sem **WHERE** daria **UPDATE** partituras **SET** preco = 35.00 (mudava todas!)

LIKE '%S%' encontra qualquer coisa que contenha 'S'

LIKE 'S%' encontra: palavras que contenham 'S' no início.

BETWEEN 1750 **AND** 1880 inclui o que fica dentro do rango.

Teste sempre com **SELECT** antes de **DELETE**

```
SELECT * FROM partituras WHERE ano_composicao BETWEEN 1750 AND 1880;
```

```
DELETE FROM partituras WHERE ano_composicao BETWEEN 1750 AND 1880;
```